

Services have an integrated load balancer that distributes network traffic to all pods of an exposed deployment. Services continuously monitor the running pods using endpoints, to ensure the traffic is sent only to available pods.

Rolling updates allow updates of deployments to take place with zero downtime by incrementally updating pods instances with new ones. These allow the following actions:

- Promote an application from one environment to another (via container image updates)
- Rollback to previous versions
- Continuous integration and continuous delivery of applications with zero downtime

Managing Kubernetes applications

Kubernetes application management includes deployment and operations. Given below is a list of operational challenges:

- Hardening and compliance
- Managing configurations of all workloads deployed on the cluster
- Managing multi-tenant clusters
- Balancing security and agility

Deployment on a hosted Kubernetes

A pod is the basic execution unit of a Kubernetes application. Each pod represents a part of a workload that is running on a cluster. Type *kubectl* in a terminal to see its usage, and this performs the specified action (like create, describe) on the specified resource (like node, container).

```
$ kubectl
```

To view the nodes in the cluster, run the *kubectl get nodes* command. You can see the available nodes (available node name: *minikube*). Kubernetes will choose where to deploy our application based on available node resources:

```
$ kubectl get nodes
NAME      STATUS    ROLES          AGE   VERSION
minikube  Ready     control-plane  19m   v1.20.2
```

To deploy the first app on Kubernetes with the *kubectl create deployment* command, it is necessary to provide the deployment name and app image location (include the full repository URL for images hosted outside the Docker hub):

```
$ kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
```

To list your deployments, use the *kubectl get deployments* command. We can see there is only one deployment running a single instance of the app. The instance is running inside a Docker container on a node.

```
$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp 1/1     1             1           23m
```

There are other options for exposing your application outside the Kubernetes cluster. To open a second terminal window to run the proxy, type *kubectl proxy* in a terminal. The proxy enables direct access to the API from these terminals:

```
Terminal 2 +
$ echo -e "\n\n\n[92mStarting Proxy.
After starting it we first Terminal Tab\n"; Please click the
Starting Proxy. After starting it will not output a response.
Please click the first Terminal Tab
$ kubectl proxy
Starting to serve on 127.0.0.1:8001
```

We can query the version directly through the API using the *curl* command.

Note: Check the top of the terminal. The proxy runs in a new tab (Terminal 2), and the recent commands were executed in the original tab (Terminal). The proxy still runs in the second tab, and this allows your *curl* command to work using *localhost:8001*. If port 8001 is not accessible, ensure that the *kubectl proxy* started above is running.

```
Terminal 2 +
Kubernetes Bootcamp Terminal
$ sleep 1; launch.sh
Starting Kubernetes. This is expected to take less than a minute....
Kubernetes Started
$
$
$ curl http://localhost:8001/version
{
  "major": "1",
  "minor": "20",
  "gitVersion": "v1.20.2",
  "gitCommit": "faecb196815e248d3ecfb03c680a4507229c2a56",
  "gitTreeState": "clean",
  "buildDate": "2021-01-13T13:20:00Z",
  "goVersion": "go1.15.5",
  "compiler": "gc",
  "platform": "linux/amd64"
```

The road ahead

Since Kubernetes is open source, it allows the freedom to take advantage of on-premises, hybrid, or public cloud infrastructures. It lets you effortlessly move workloads to where it matters to you. Kubernetes can be combined with modern pipeline tools like Jenkins X to build highly agile and lean CI/CD (continuous integration and continuous delivery/continuous deployment) systems. Modern, high-performance DevOps teams should make good use of Kubernetes. **END** 🐼

By: Dr S. Balakrishnan and Arun R.

Dr S. Balakrishnan is the professor and head, Department of Computer Science and Business System, Sri Krishna College of Engineering and Technology, Coimbatore.

Arun R. is an assistant professor for CSE, Hindusthan Institute of Technology, Coimbatore.